



JAVA PROGRAMMING COURSE

Introduction to

JAVA PROGRAMMING

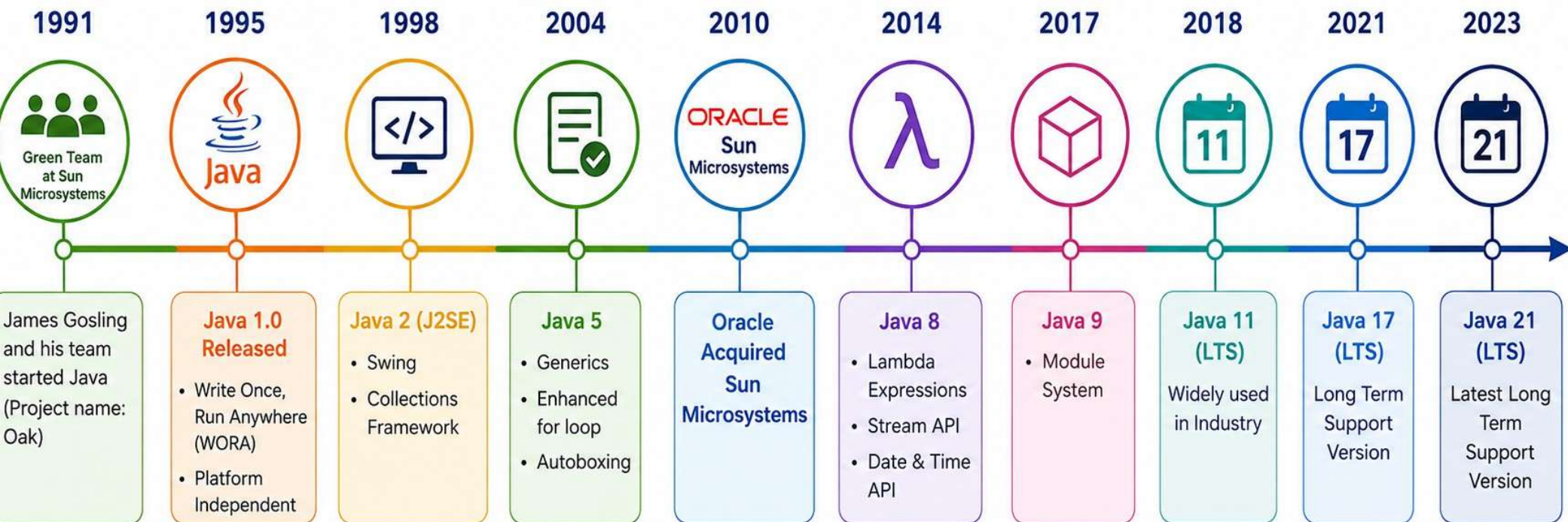
Dr. R. Sahila Devi | Department of CSE | Rohini College of Engineering and Technology



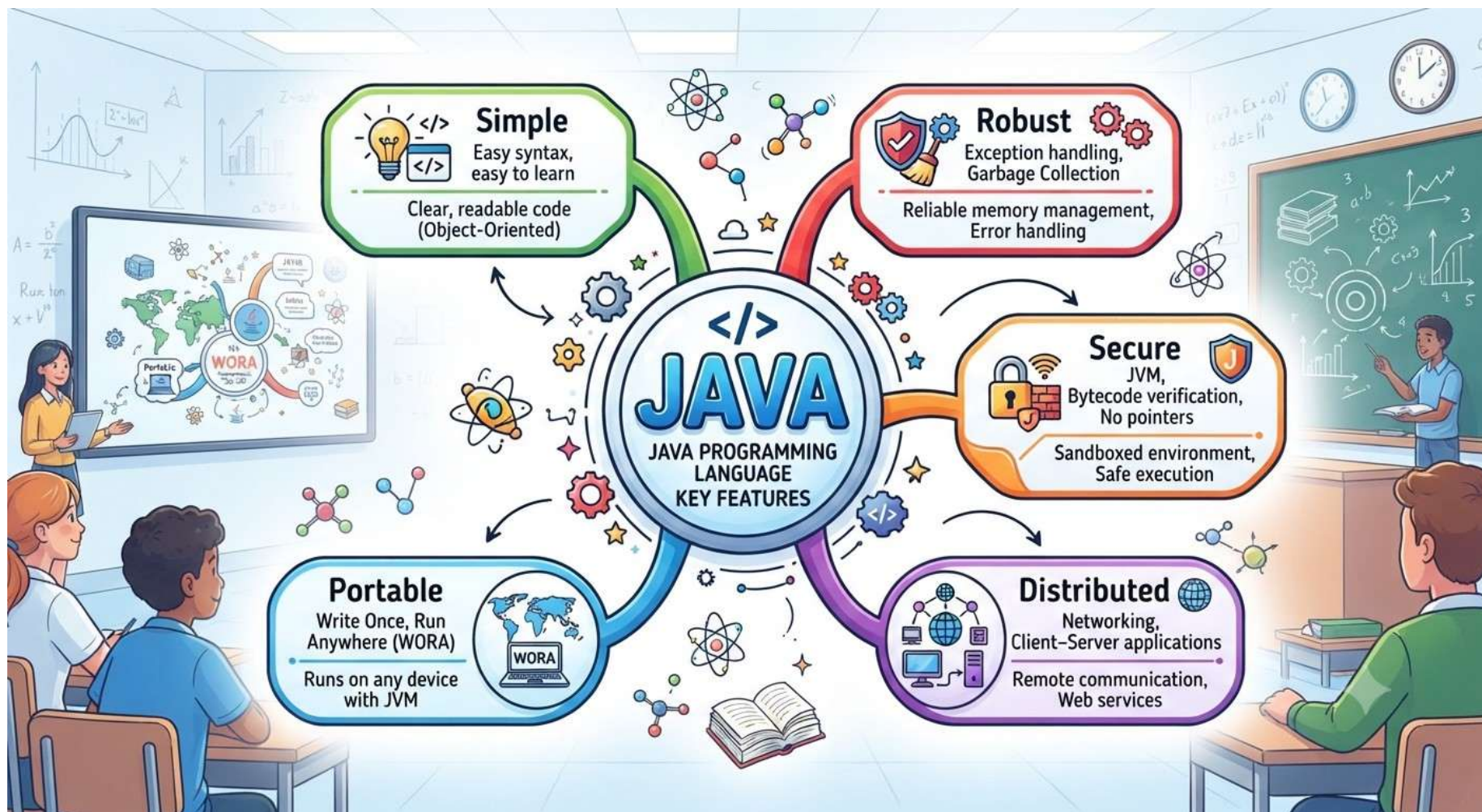
Unit -1

Revision at a Glance

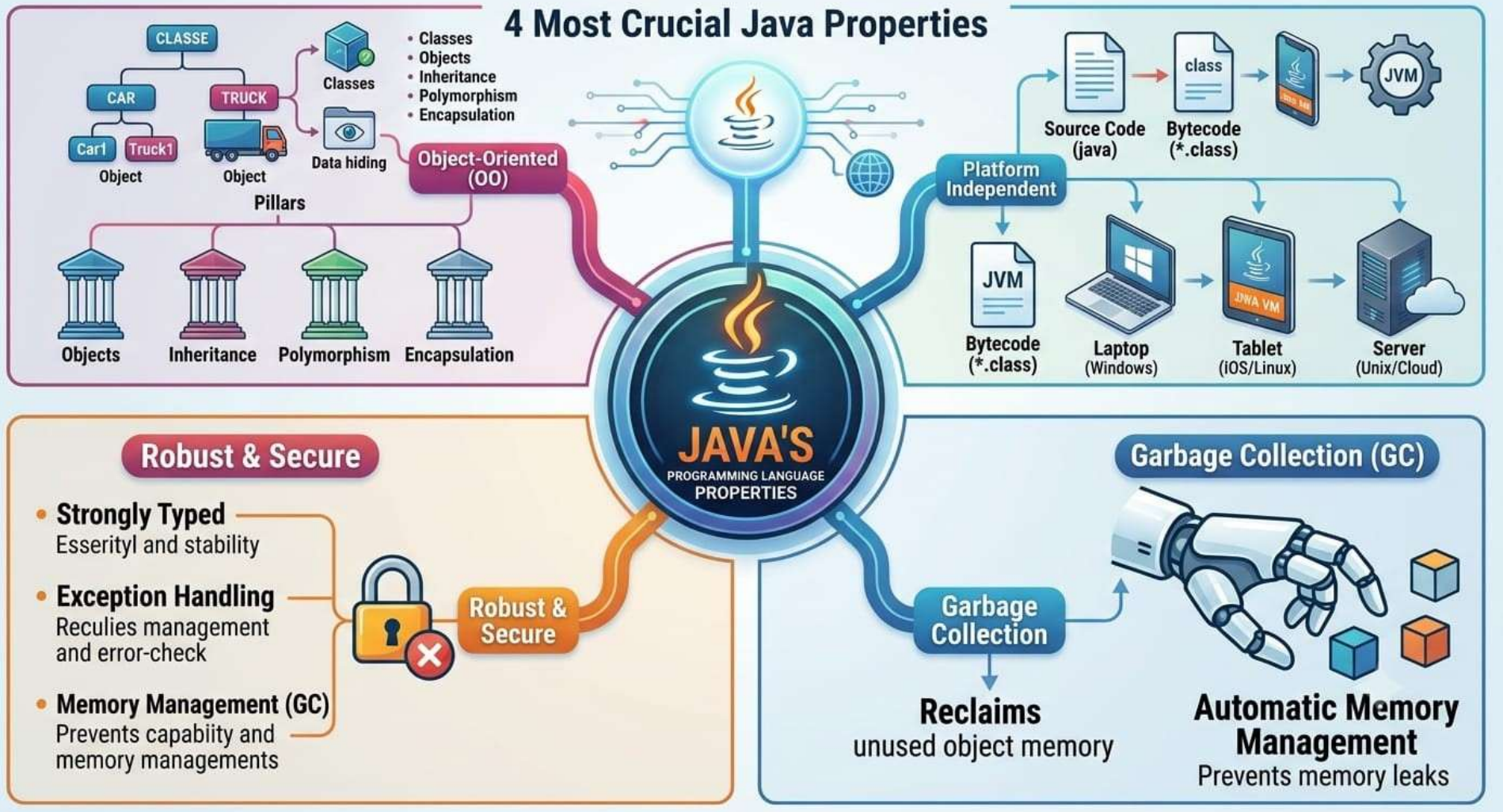
THE HISTORY OF JAVA



Java is platform independent, secure, robust and widely used for Web, Mobile, Enterprise and many other applications.



4 Most Crucial Java Properties



HOW JAVA WORKS: CODE TO EXECUTION (COMPLETE WORKFLOW)

STAGE 1: DEVELOPMENT & COMPILATION

Java compiles **Source Code** once to **Universal Bytecode**, not Machine Code.

Source Code
(MyProgram.java)

Compiler
(javac)

WRITE ONCE

Java Bytecode
(.class)

Standard
Java
Bytecode

JVM is an interface:
Standard Bytecode goes
in, **Platform Machine
Code** comes out.

JIT Compiler optimizes
performance by
translating hot
bytecode on the fly.

STAGE 3: PLATFORM-SPECIFIC EXECUTION

A
Windows PC Platform

Windows-Specific JVM
(interpreter)

Windows-Specific Machine Code

B
macOS Platform

macOS-Specific JVM

macOS-Specific Machine Code

C
Linux LINUX SERVER PLATFORM

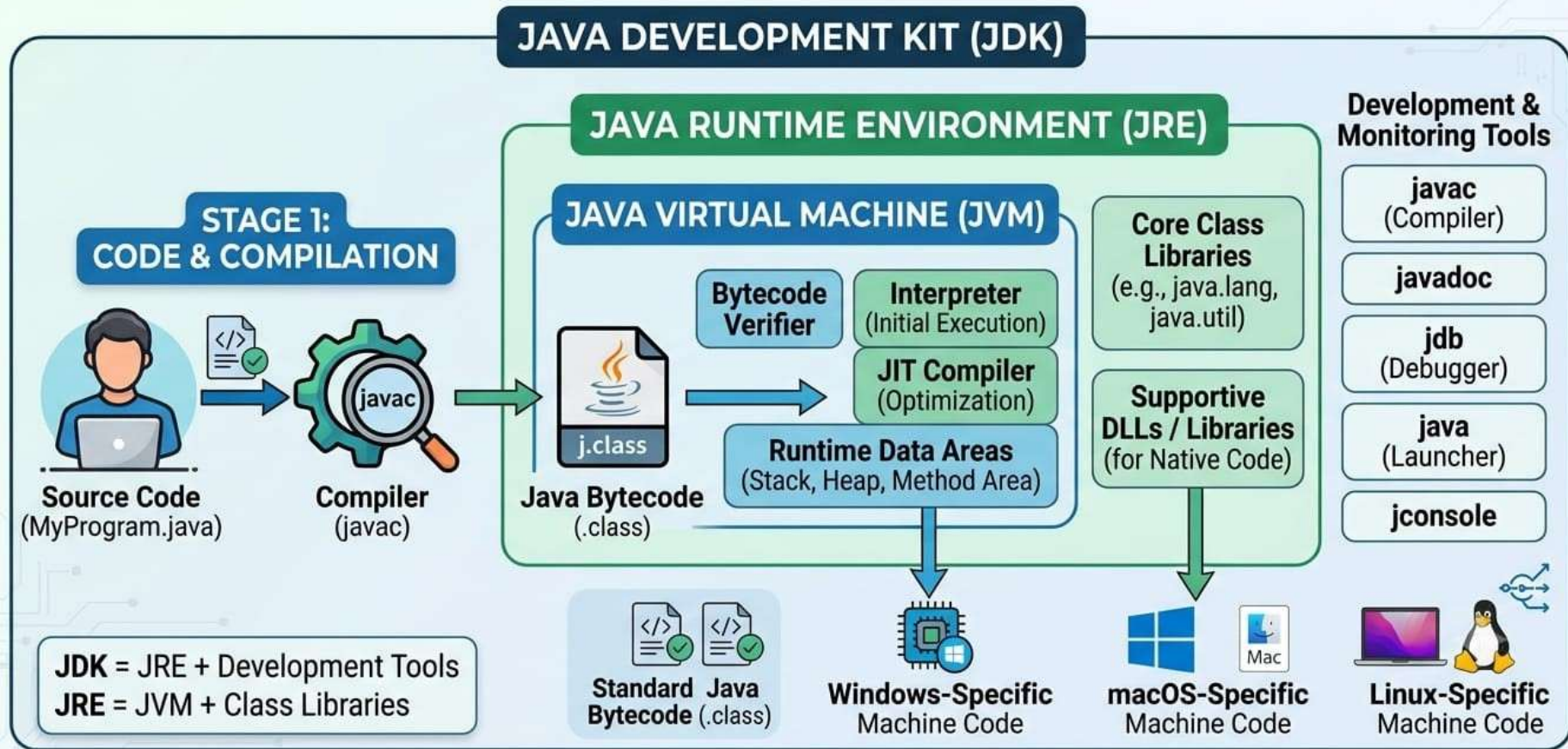
Linux-Specific JVM

Linux-Specific Machine Code

RUN AREM

ANYWHERE

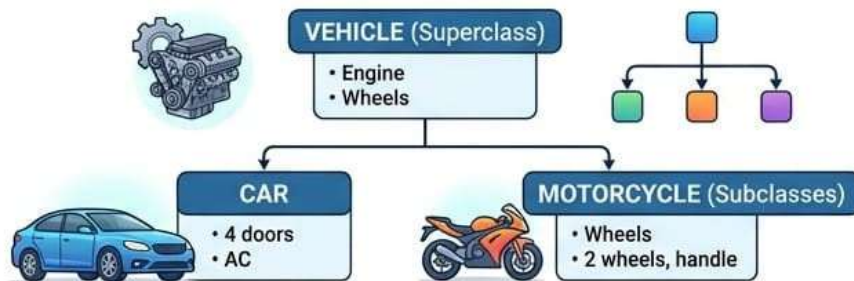
THE JAVA ECOSYSTEM: HIERARCHY & ARCHITECTURE



UNDERSTANDING THE 4 PILLARS OF OOP with REAL-WORLD EXAMPLES

1. INHERITANCE (Hierarchy)

Subclasses derive properties/behaviors from a parent class.



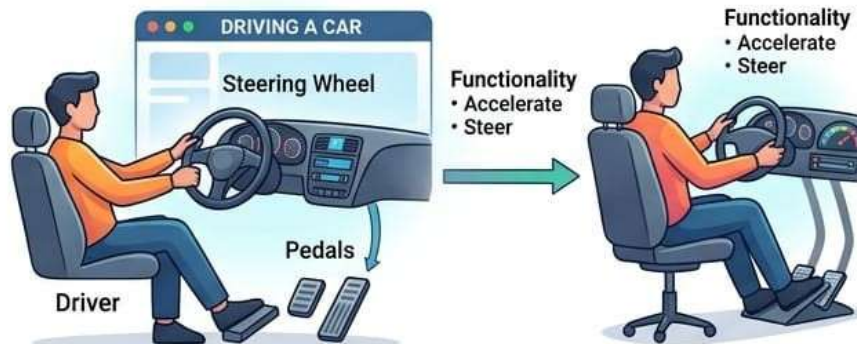
2. ENCAPSULATION (Data Hiding)

Binding data (Attributes) and methods (Functions) within a class, hiding inner details.



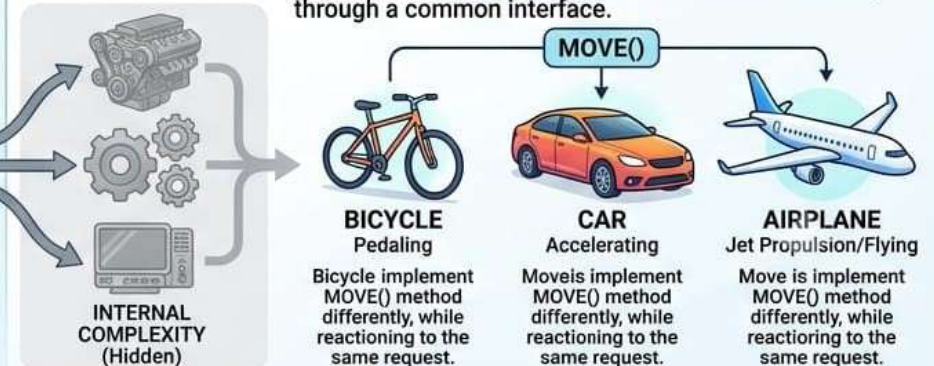
3. ABSTRACTION (Simplification)

Exposing essential features while hiding complex implementation details.

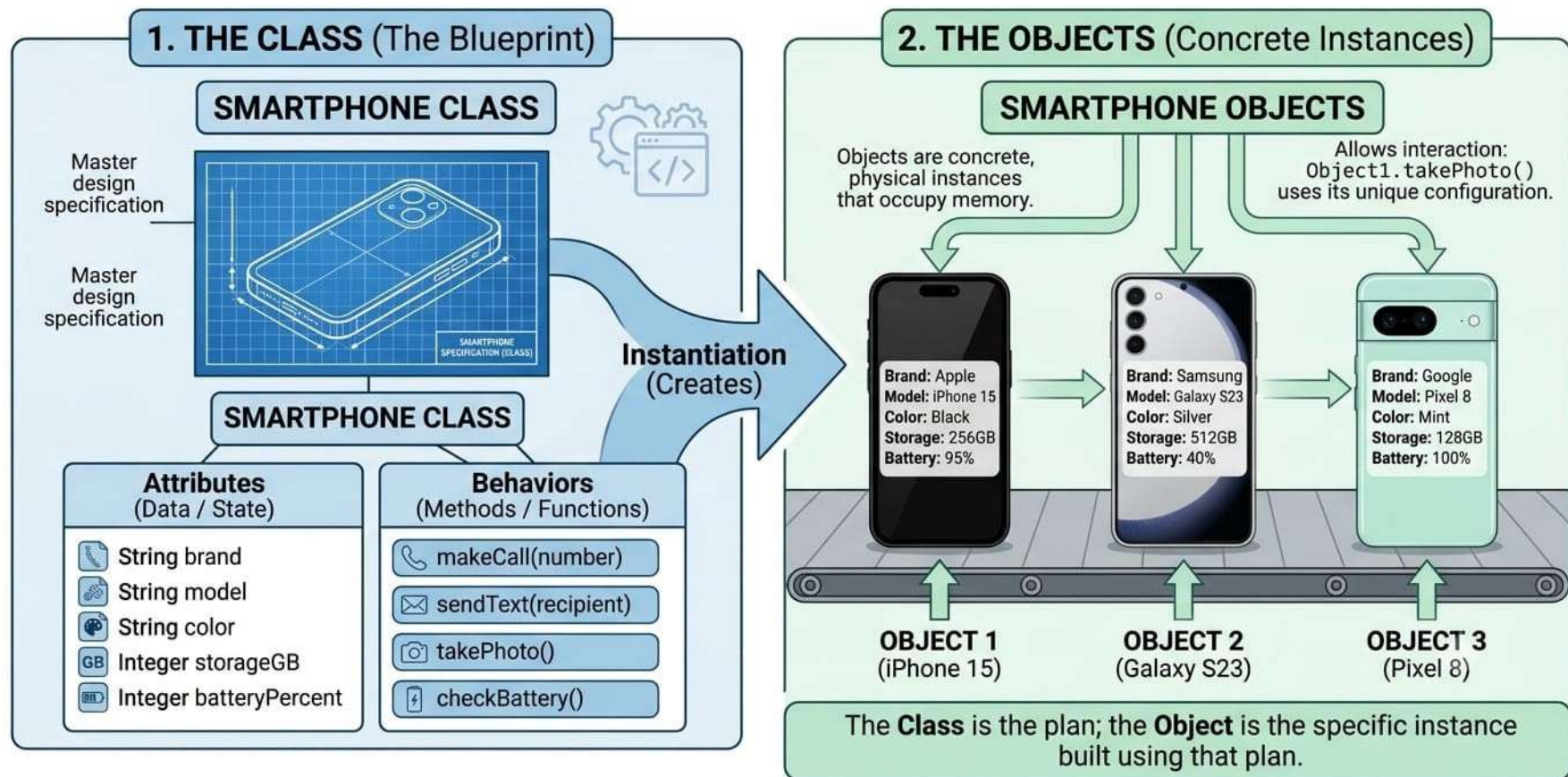


4. POLYMORPHISM (Many Forms)

Allowing different object types to be treated uniformly through a common interface.



CLASS vs. OBJECT: BLUEPRINT to REALITY



WHY IS JAVA PLATFORM INDEPENDENT?

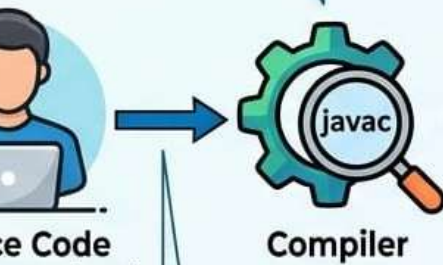
The Role of Bytecode and the Java Virtual Machine (JVM)

STAGE 1: COMPILATION (WRITE ONCE)

STAGE 2: EXECUTION (RUN ANYWHERE)

Java compiles **Source Code** once to **Universal Bytecode**, not **Machine Code**.

The key: **Standard Bytecode** is generated on any platform.



Java compiles **Source Code** once to **Universal Bytecode**, not **Machine Code**.

The **JVM** acts as a **platform-specific** **lform-dependent**; the **Bytecode** is **standard**.

Standard
Java
Bytecode



Windows PC
Platform



macOS Platform

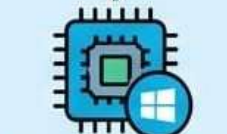


Linux Platform

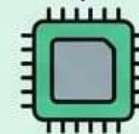
Windows-
Specific JVM
(interpreter)

macOS-Specific
JVM

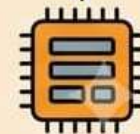
Linux-Specific
JVM



Machine Code
(Windows x86)



Machine Code
(Apple Silicon/ARM)



Machine Code
(Linux ARM/x86)

WMM



JAVA DATA TYPES: A COMPREHENSIVE OVERVIEW

PRIMITIVE DATA TYPES

Store simple, fixed-size data values directly

Numeric Types (Integer)

- byte** (8-bit) $\leftrightarrow$ -128
- short** (16-bit)
- int** (32-bit) $\rightarrow$ -128
- long** (64-bit) $\rightarrow$ -128 to 127 to 127

Numeric Types (Floating-Point)

- float** (32-bit) precision - 32 mi
- double** (64-bit) precision - decimals

Character Type

char (16-bit)

A 文 unicode

Boolean Type

boolean (1-bit*)

TRUE FALSE

NON-PRIMITIVE DATA TYPES (Reference Types)

Store references (memory addresses) to objects. Created by programmers

String

Text string
Hello World!

Array

Sequential collection
[e.g., value, 1, 3, 4, ...]

point to memory locations)

Class

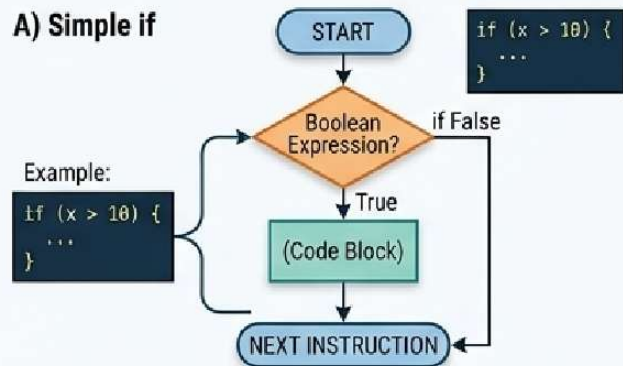
class Person

Interface

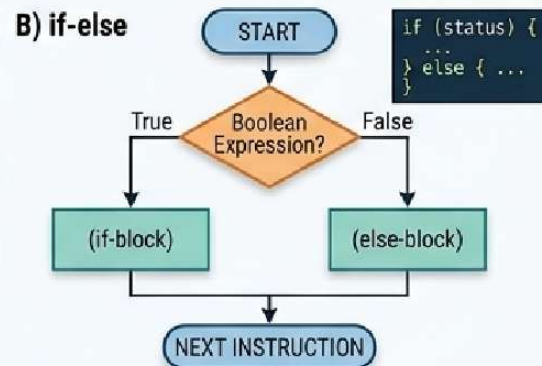
JAVA DECISION MAKING: FLOW AND SYNTAX

I. FUNDAMENTAL "IF" STRUCTURES

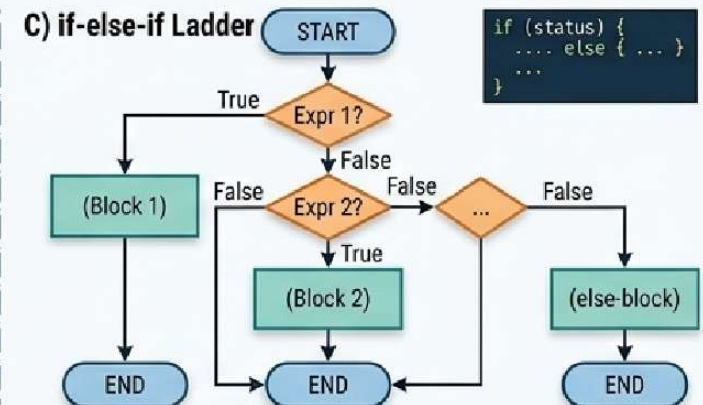
A) Simple if



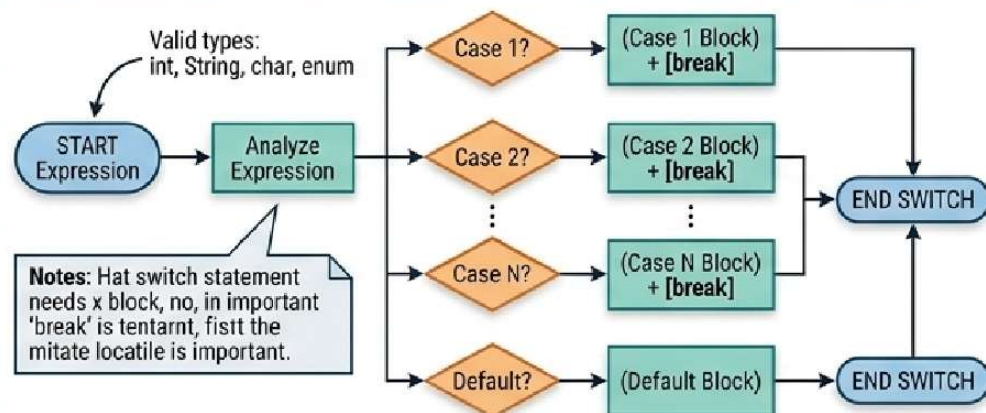
B) if-else



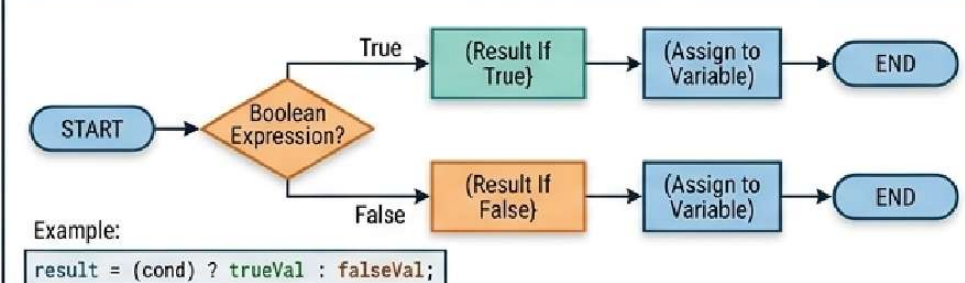
C) if-else-if Ladder



II. MULTIPLE ALTERNATIVE ("SWITCH")



III. TERNARY OPERATOR ('? :')

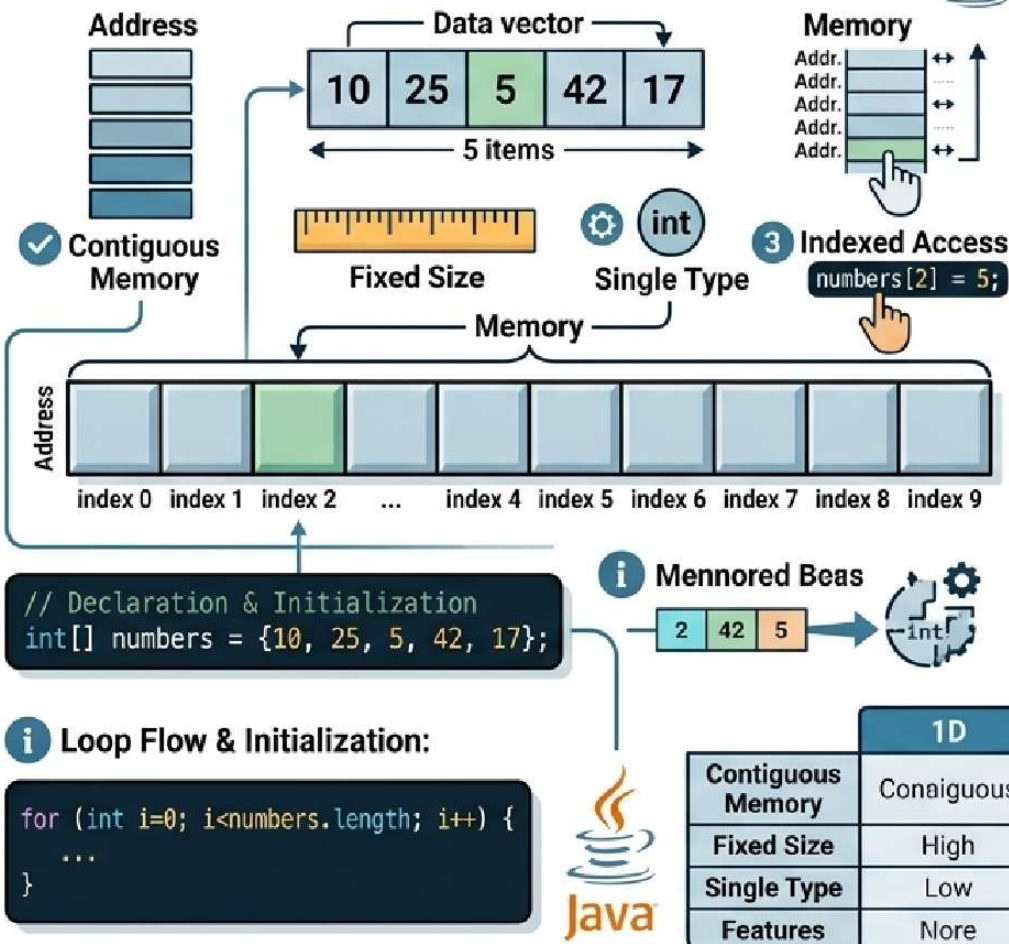


LEGEND

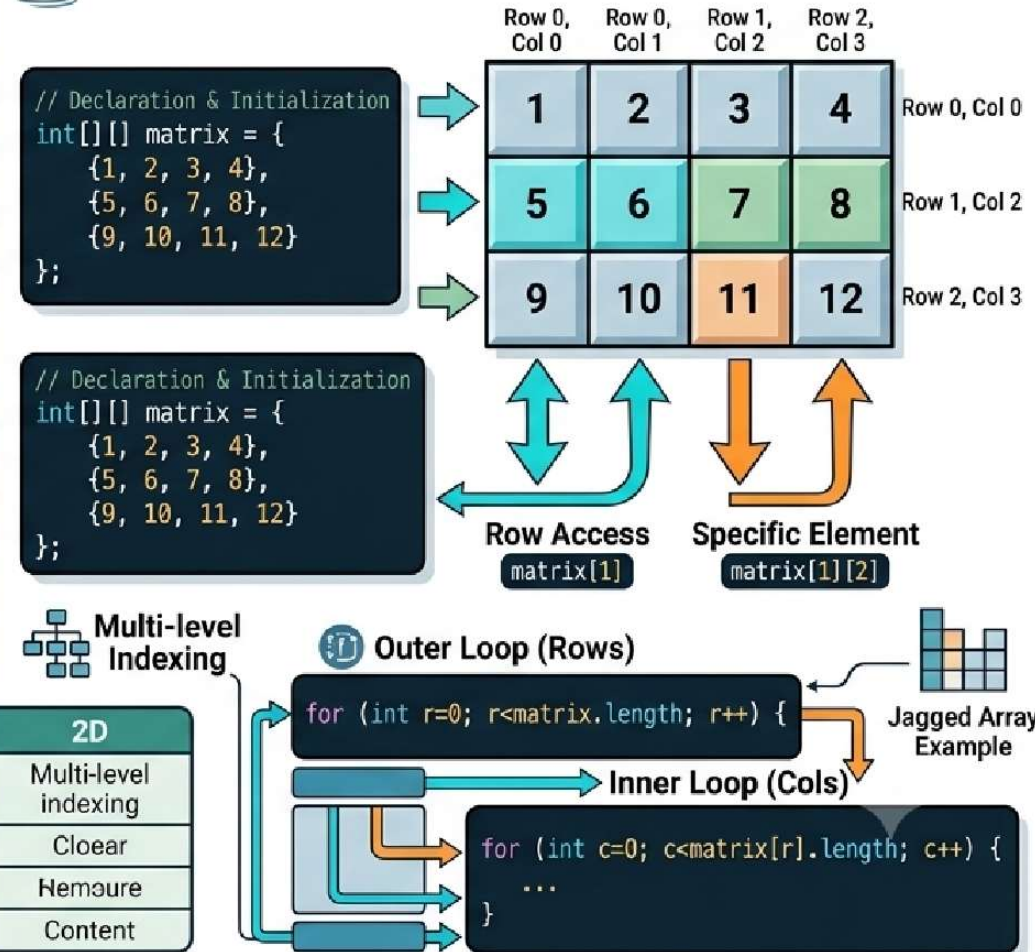


JAVA ARRAYS: STRUCTURE AND USAGE (1D & 2D)

I. ONE-DIMENSIONAL ARRAY (1D)



II. TWO-DIMENSIONAL ARRAY (2D)



JAVA ACCESS SPECIFIERS

(Real World Example - Student)

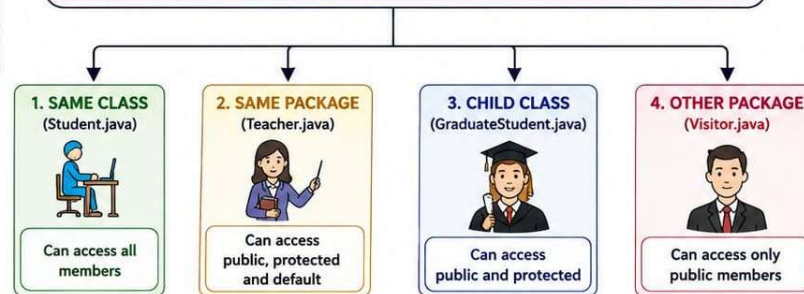
REAL WORLD ANALOGY

-  **Public**
= School Notice Board
(Everyone can see)
-  **Protected**
= Staff Room
(Teachers & Authorized Members)
-  **Default**
= Classroom
(Only students of that class)
-  **Private**
= Student's Diary
(Only the student)

Student		
 public	name : String	Accessible by everyone
 protected	rollNo : int	Accessible in same package and subclasses
 default	section : String	Accessible in same package only
 private	password : String	Accessible only within Student class

LEGEND

-  **public**
-  **protected**
-  **default**
-  **private**



ACCESSIBILITY SUMMARY				
Access Specifier	Same Class (Student.java)	Same Package (Teacher.java)	Child Class (GraduateStudent.java)	Other Package (Visitor.java)
public	✓ Yes	✓ Yes	✓ Yes	✓ Yes
protected	✓ Yes	✓ Yes	✓ Yes	✗ No
default	✓ Yes	✓ Yes	✗ No	✗ No
private	✓ Yes	✗ No	✗ No	✗ No

KEY TAKEAWAY

The more restrictive the access modifier, the fewer places it can be accessed.

Order of Restriction

public (least)
↓
protected
↓
default
↓
private (most)



Real Coding Demonstrations

Hands-on Java examples: variables, classes & objects, loops and arrays

DEMO 1

Hello World & Variables



```
public class HelloJava {  
    public static void main(String[] args) {  
        String name = "World";  
        int year = 2026;  
        double version = 21.0;  
  
        System.out.println(  
            "Hello, " + name + "!");  
        System.out.println(  
            "Java version: " + version);  
    }  
}
```

CONSOLE OUTPUT

```
Hello, World!  
Java version: 21.0
```

KEY TAKEAWAYS

- `main()` is the program's entry point
- `String`, `int`, `double` are common types
- `+` concatenates strings with values
- `println()` prints with a line break

DEMO 2

Class & Object in Action

```
class Student {  
    private String name;  
    private int marks;  
  
    Student(String name, int marks) {  
        this.name = name;  
        this.marks = marks;  
    }  
  
    void showResult() {  
        String status =  
            (marks >= 40) ? "Pass" : "Fail";  
        System.out.println(  
            name + ": " + status);  
    }  
}
```

CALLING CODE

```
Student s1 =  
    new Student("Ravi", 78);  
s1.showResult();
```

OUTPUT

Ravi: Pass

The class is a blueprint; "new" creates a real object in memory with its own name and marks.

DEMO 3

Loops & Arrays

```
public class SumArray {  
    public static void main(String[] args) {  
        int[] scores = {88, 92, 76, 64, 95};  
        int total = 0;  
  
        for (int i = 0; i < scores.length; i++) {  
            total += scores[i];  
        }  
  
        double avg = total / (double) scores.length;  
        System.out.println("Average: " + avg);  
    }  
}
```

OUTPUT

Average: 83.0

HOW IT WORKS

- scores.length gives the array size (5)
- The for loop visits each index 0–4
- total += scores[i] accumulates the sum
- (double) forces decimal division for avg



Knowledge Check

Five quick questions to test what we've covered so far

KNOWLEDGE CHECK — Questions 1–3

Q1

Which keyword is used to create an object in Java?

A. class

B. new

C. public

D. static

Q2

Which of these is a non-primitive Java data type?

A. int

B. boolean

C. String

D. char

Q3

What does WORA stand for?

A. Write Once, Run Anywhere

B. Write Once, Run Always

C. Web Object Runtime API

D. Write Only, Read Anywhere

KNOWLEDGE CHECK — Questions 4–5

Q4

Which access specifier is the most restrictive?

A. public

B. protected

C. default

D. private

Q5

In a for loop over an array, which gives the number of elements?

A. array.size()

B. array.length

C. array.count

D. array.len()

Q & A

Questions, thoughts, or something you'd like demoed again?

Thank You

Dr. R. Sahila Devi

Department of CSE · Rohini College of Engineering and Technology